

Chapter 11. Event-Driven Batch Processing

In the previous chapter, we saw a generic framework for work queue processing, as well as a number of example applications of simple work queue processing. Work queues are great for enabling individual transformations of one input to one output. However, there are a number of batch applications where you want to perform more than a single action, or you may need to generate multiple different outputs from a single data input. In these cases, you start to link work queues together so that the output of one work queue becomes the input to one or more other work queues, and so on. This forms a series of processing steps that respond to events, with the events being the completion of the preceding step in the work queue that came before it.

These sort of event-driven processing systems are often called *workflow* systems, since there is a flow of work through a directed, acyclic graph that describes the various stages and their coordination. A basic illustration of such a system is shown in [Figure 11-1](#).

The most straightforward application of this type of system simply chains the output of one queue to the input of the next queue. But as systems become more complicated there are a series of different patterns that emerge for linking a series of work queues together. Understanding and designing in terms of these patterns is important for comprehending how the system is working. The operation of an event-driven batch processor is similar to event-driven FaaS. Consequently, without an overall blueprint for how the different event queues relate to each other, it can be hard to fully understand how the system is operating.

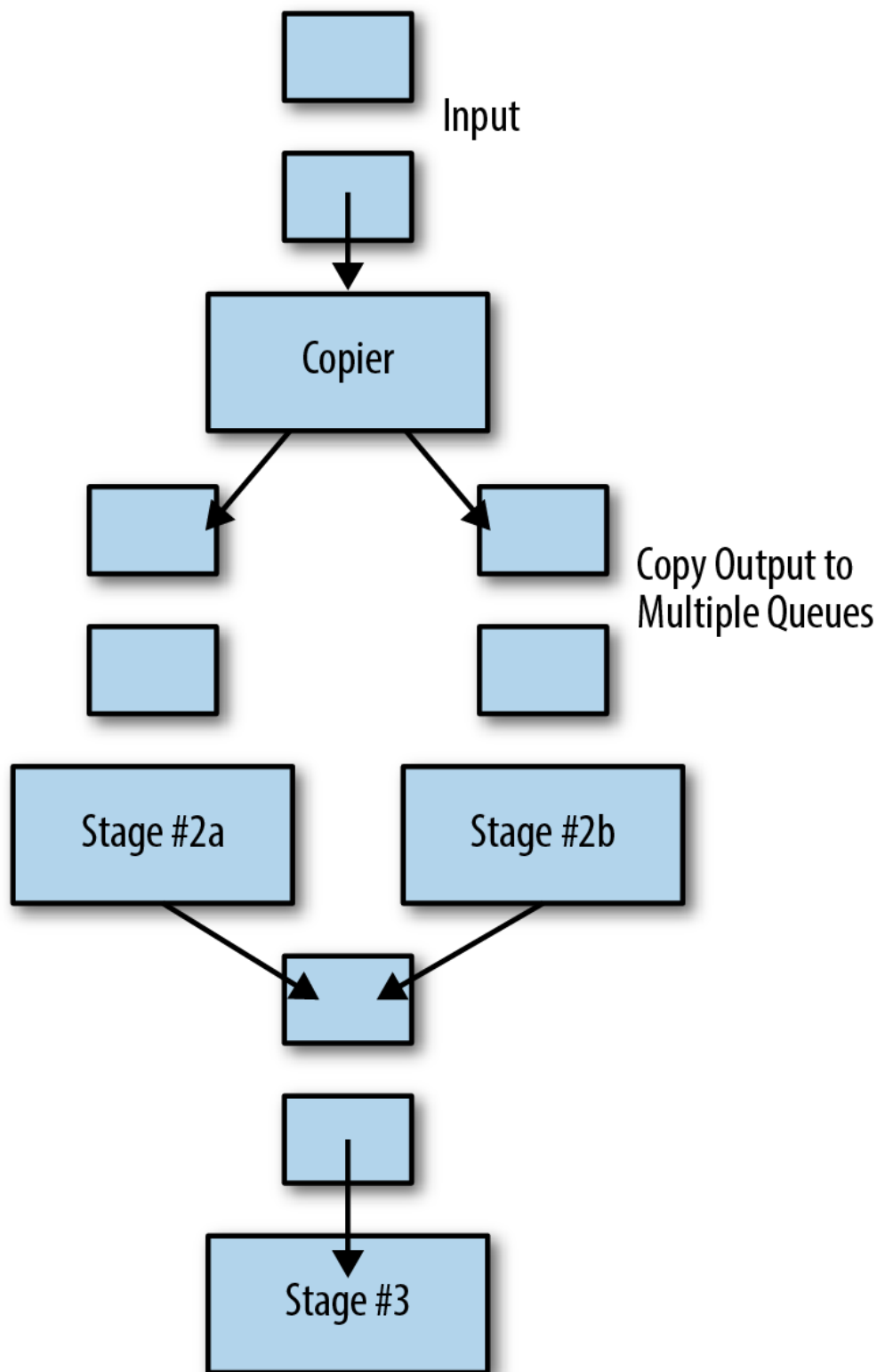


Figure 11-1. This workflow combines copying work into multiple queues (Stage 2a, 2b) parallel processing of those queues, and combining the result back into a single queue (Stage 3)

Patterns of Event-Driven Processing

Beyond the simple work queue described in the previous chapter, there are a number of patterns for linking work queues together. The simplest pattern—one where the output of a single queue becomes the input to a second queue—is straightforward enough that we won't cover it here. We will describe patterns that involve the coordination of multiple different queues or the modification of the output of one or more work queues.

Copier

The first pattern for coordinating work queues is a copier. The job of a copier is to take a single stream of work items and duplicate it out into two or more identical streams. This pattern is useful when there are multiple different pieces of work to be done on the same work item. An example of this might be rendering a video. When rendering a video, there are a variety of different formats that are useful depending on where the video is intended to be shown. There might be a 4K high-resolution format for playing off of a hard drive, a 1080-pixel rendering for digital streaming, a low-resolution format for streaming to mobile users on slow networks, and an animated GIF thumbnail for displaying in a movie-picking user interface. All of these work items can be modeled as separate work queues for each render, but the input to each work item is identical. An illustration of the copier pattern applied to transcoding is shown in [Figure 11-2](#).

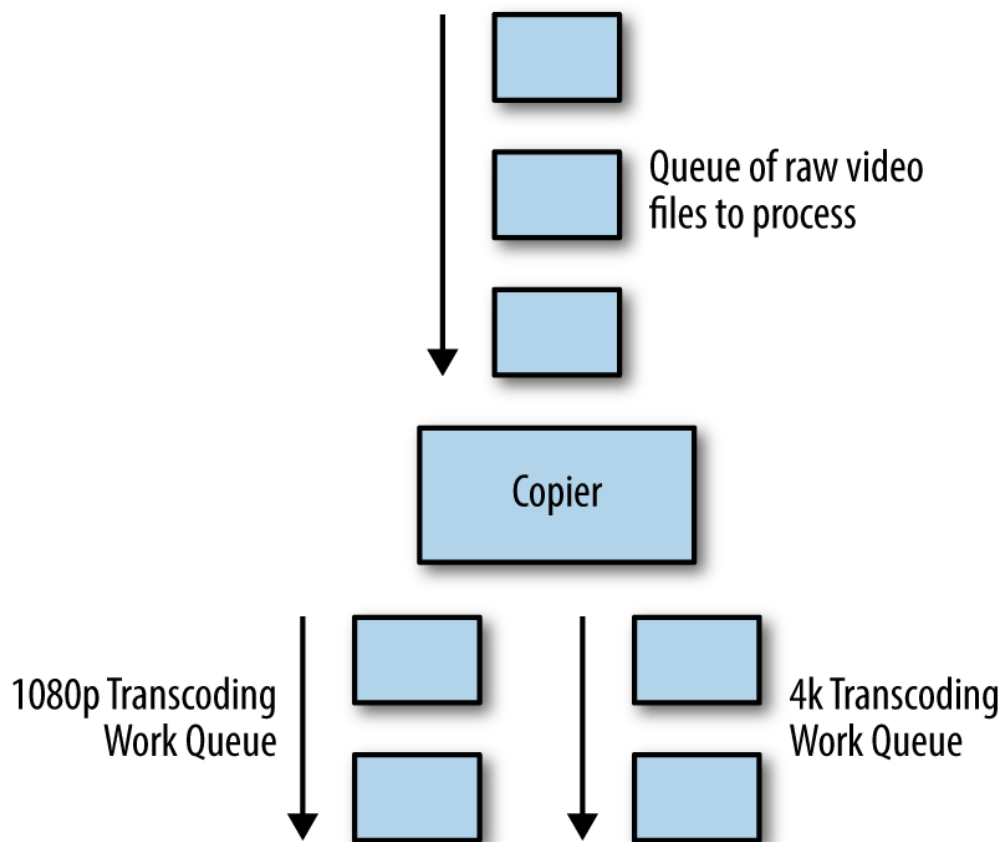


Figure 11-2. The copier batch pattern for transcoding

Filter

The second pattern for event-driven batch processing is a filter. The role of a filter is to reduce a stream of work items to a smaller stream of work items by filtering out work items that don't meet particular criteria. As an example of this, consider setting up a batch workflow that handles new users signing up for a service. Some set of those users will have ticked the checkbox that indicates that they wish to be contacted via email for pro-

motions and other information. In such a workflow, you can filter the set of newly signed-up users to only be those who have explicitly opted into being contacted.

Ideally you would compose a filter work queue source as an ambassador that wraps up an existing work queue source. The original source container provides the complete list of items to be worked on, and the filter container then adjusts that list based on the filter criteria and only returns those filtered results to the work queue infrastructure. An illustration of this use of the adapter pattern is shown in [Figure 11-3](#).

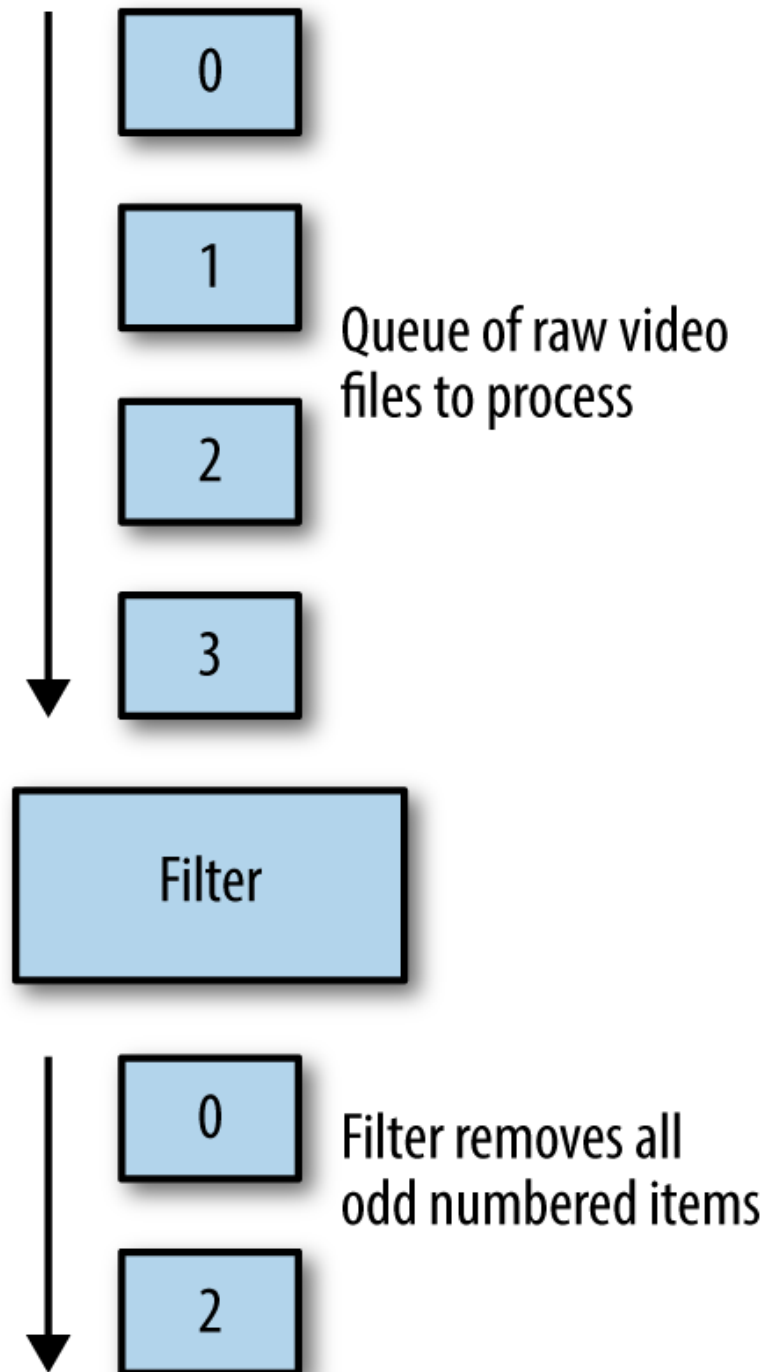


Figure 11-3. An example of a filter pattern that removes all odd-numbered work items

Splitter

Sometimes you don't want to just filter things out by dropping them on the floor, but rather you have two different kinds of input present in your set of work items and you want to divide them into two separate work queues without dropping any of them. For this task, you want to use a splitter. The role of a splitter is to evaluate some criteria—just like a filter—but instead of eliminating input, the splitter sends different inputs to different queues based on that criteria.

An example of an application of the splitter pattern is processing online orders where people can receive shipping notifications either by email or text message. Given a work queue of items that have been shipped, the splitter divides it into two different queues: one that is responsible for sending emails and another devoted to sending text messages. A splitter can also be a copier if it sends the same output to multiple queues, such as when a user selects both text messages and email notifications in the previous example. It is interesting to note that a splitter can actually also be implemented by a copier and two different filters. But the splitter pattern is a more compact representation that captures the job of the splitter more succinctly. An example of using the splitter pattern to send shipping notifications to users is shown in [Figure 11-4](#).

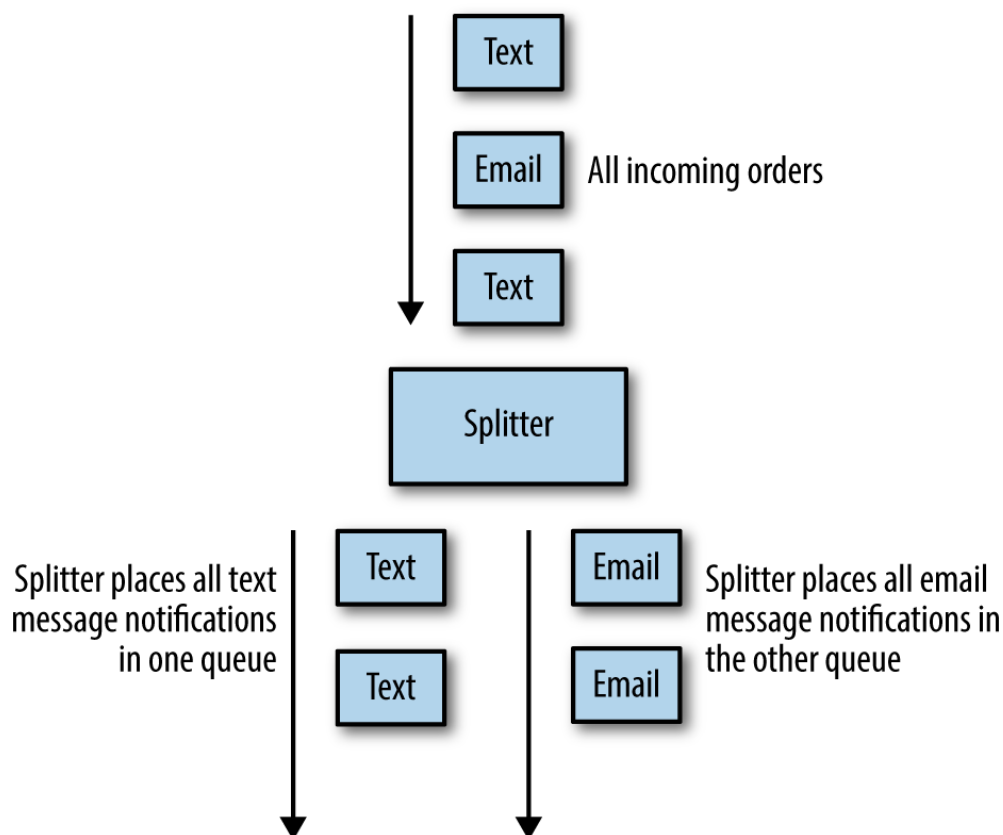


Figure 11-4. An example of the batch splitter pattern splitting shipping notifications into two different queues

Sharder

A slightly more generic form of splitter is a sharder. Much like the sharded server that we saw in earlier chapters, the role of a sharder in a work-

flow is to divide up a single queue into an evenly divided collection of work items based upon some sort of *sharding function*. There are several different reasons why you might consider sharding your workflow. One of the first is for reliability. If you shard your work queue, then the failure of a single workflow due to a bad update, infrastructure failure, or other problem only affects a fraction of your service.

For example, imagine that you push a bad update to your worker container, which causes your workers to crash and your queue to stop processing work items. If you only have a single work queue that is processing items, then you will have a complete outage for your service with all users affected. If, instead, you have sharded your work queue into four different shards, you have the opportunity to do a staged rollout of your new worker container. Assuming you catch the failure in the first phase of the staged rollout, sharding your queue into four different shards means that only one quarter of your users would be affected.

An additional reason to shard your work queue is to more evenly distribute work across different resources. If you don't really care which region or datacenter is used to process a particular set of work items, you can use a sharder to evenly spread work across multiple datacenters to even out utilization of all datacenters/regions. As with updates, spreading your work queue across multiple failure regions also has the benefit of providing reliability against datacenter or region failures. An illustration of a sharded queue when everything is working correctly is shown in [Figure 11-5](#).

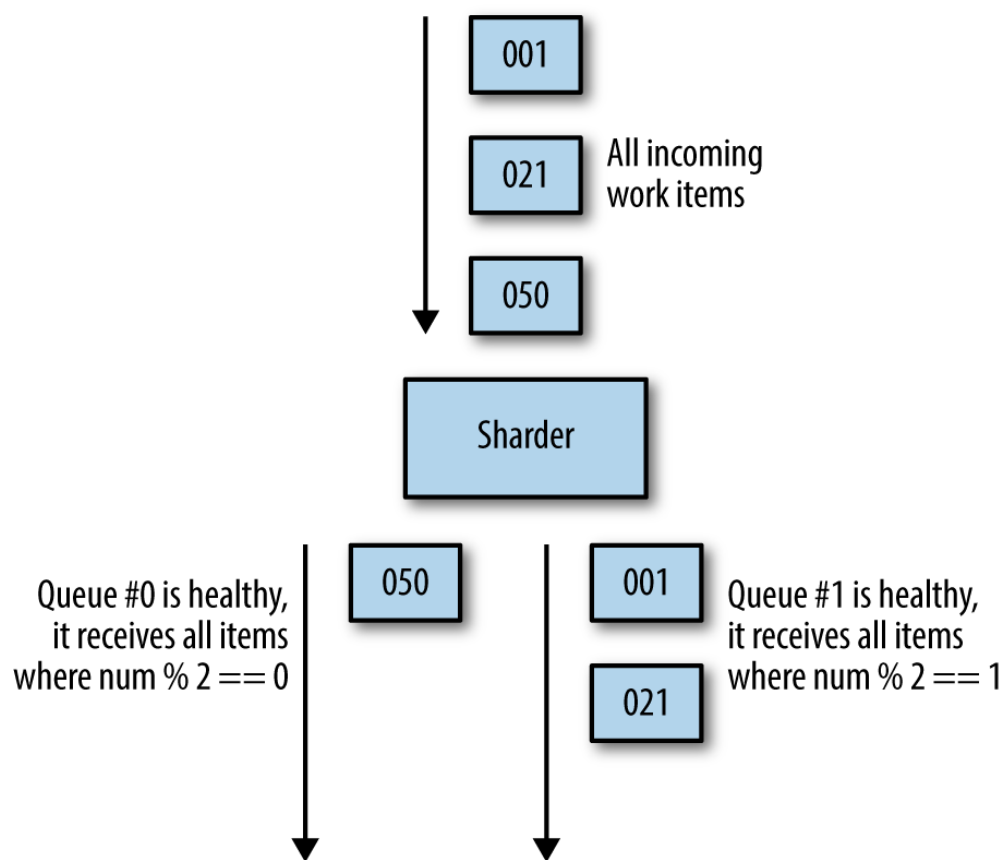


Figure 11-5. An example of the sharding pattern in a healthy operation

When the number of healthy shards is reduced due to failures, the sharding algorithm dynamically adjusts to send work to the remaining healthy work queues, even if only a single queue remains. This is illustrated in [Figure 11-6](#).

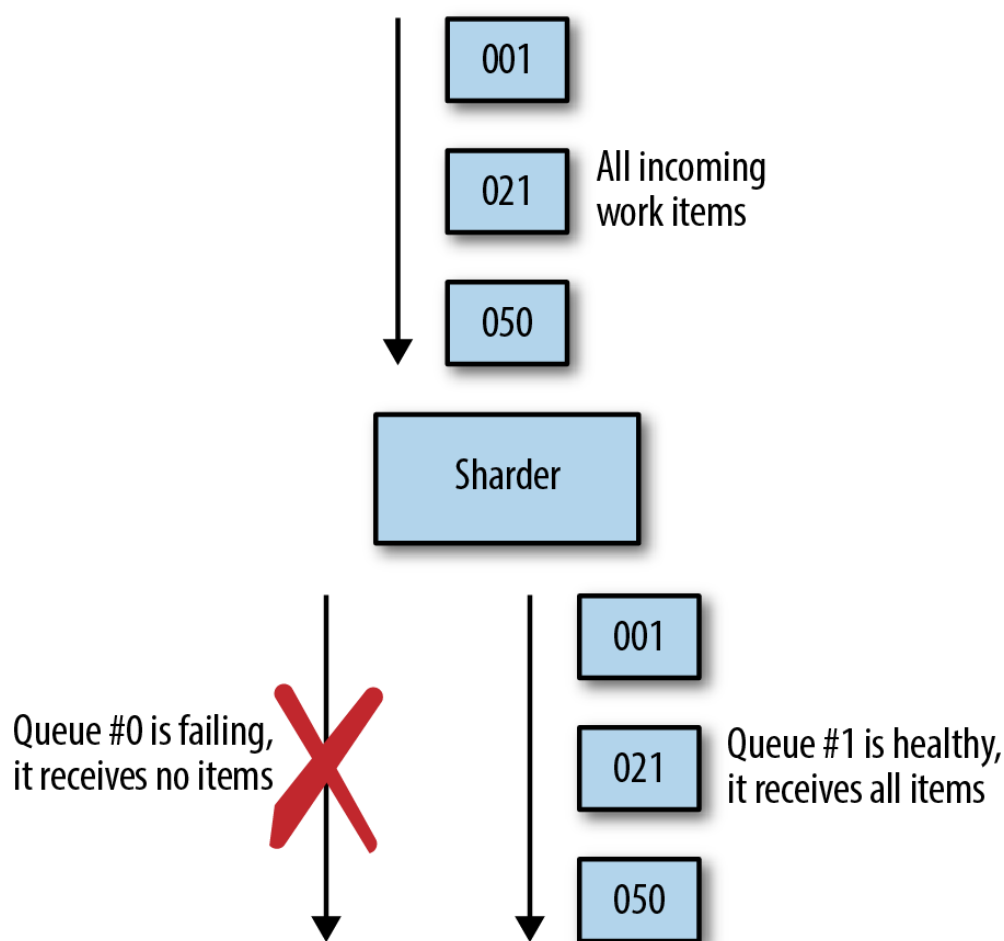


Figure 11-6. When one work queue is unhealthy the remaining work spills over to a different queue

Merger

The last pattern for event-driven or workflow batch systems is a *merger*. A merger is the opposite of a copier; the job of a merger is to take two different work queues and turn them into a single work queue. Suppose, for example, that you have a large number of different source repositories all adding new commits at the same time. You want to take each of these commits and perform a build-and-test for it. It is not scalable to create a separate build infrastructure for each source repository. We can model each of the different source repositories as a separate work queue source that provides a set of commits. We can transform all of these different work queue inputs into a single merged set of inputs using a merger adapter. This merged stream of commits is then the single source to the build system that performs the actual build. The merger is another great example of the adapter pattern, though in this case, the adapter is actually adapting multiple running source containers into a single merged source. This multi-adapter pattern is shown in [Figure 11-7](#).

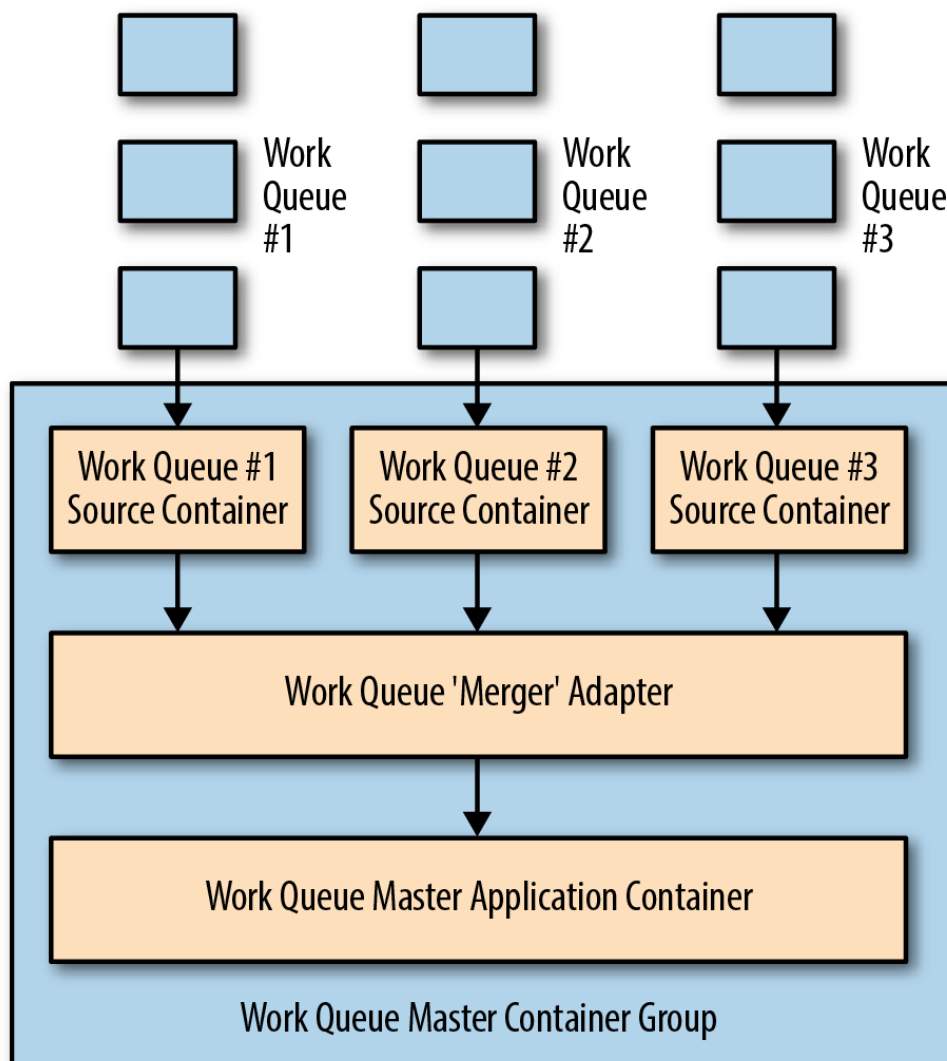


Figure 11-7. Using multiple levels of containers to adapt multiple independent work queues into a single shared queue

Hands On: Building an Event-Driven Flow for New User Sign-Up

A concrete example of a workflow helps show how these patterns can be put together to form a complete operating system. The problem this example will consider is a new-user signup flow.

Imagine that our user acquisition funnel has two stages. The first is user verification. After a new user signs up, the user then has to receive an email notification to validate their email. Once the user validates their email, they are sent a confirmation email. Then they are optionally registered for email, text message, both, or neither for notifications.

The first step in the event-driven workflow is the generation of the verification email. To achieve this reliably, we will use the shard pattern to shard users across multiple different geographic failure zones. This ensures that we will continue to process new user signups, even in the presence of partial failures. Each work queue shard sends a verification email

to the end user. At this point, this substage of the workflow is complete. This first stage of the flow is illustrated in [Figure 11-8](#).

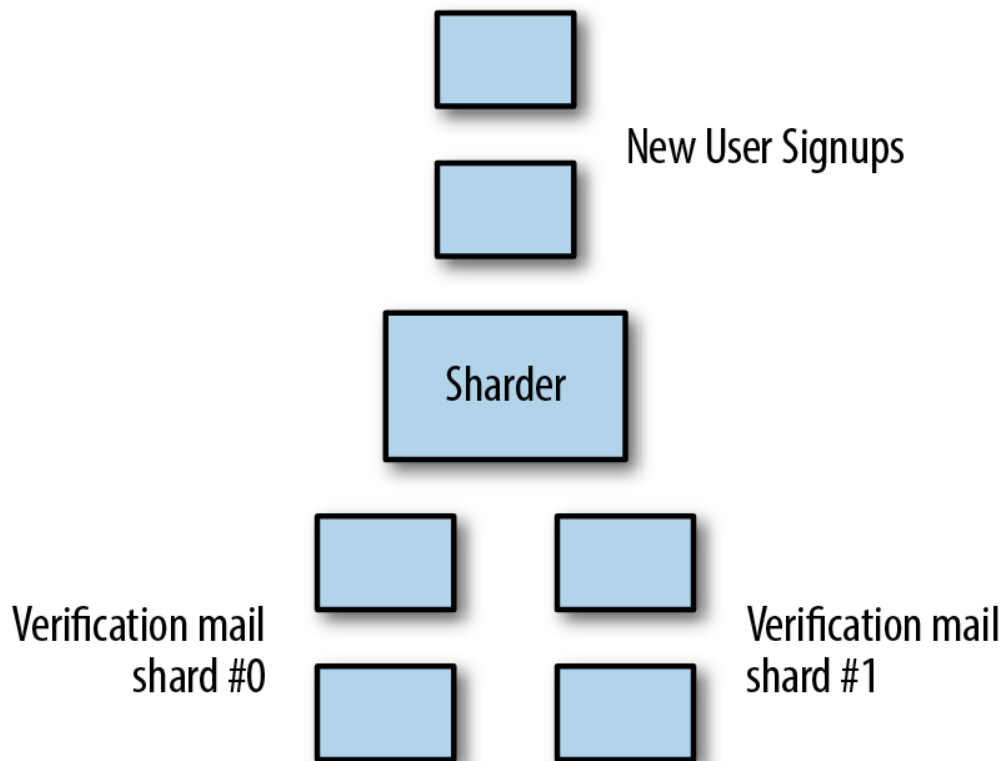


Figure 11-8. The first stage of the workflow for user sign-up

The workflow begins again when we receive a verification email from the end user. These emails become new events in a separate (but clearly related) workflow that sends welcome emails and sets up notifications. The first stage of this workflow is an example of the copier pattern, where the user is copied into two work queues. The first work queue is responsible for sending the welcome email, and the second work queue is responsible for setting up user notifications.

Once the work items have been duplicated between the queues, the email-sending queue simply takes care of sending an email message, and the workflow exits. But remember that because of the use of the copier pattern, there is still an additional copy of the event active in our workflow. This copier triggers an additional work queue to handle notification settings. This work queue feeds into an example of the filter pattern, which splits the work queue into separate email and text message notification queues. These specific queues register the user for email, text, or both notifications.

The remainder of this workflow is shown in [Figure 11-9](#).

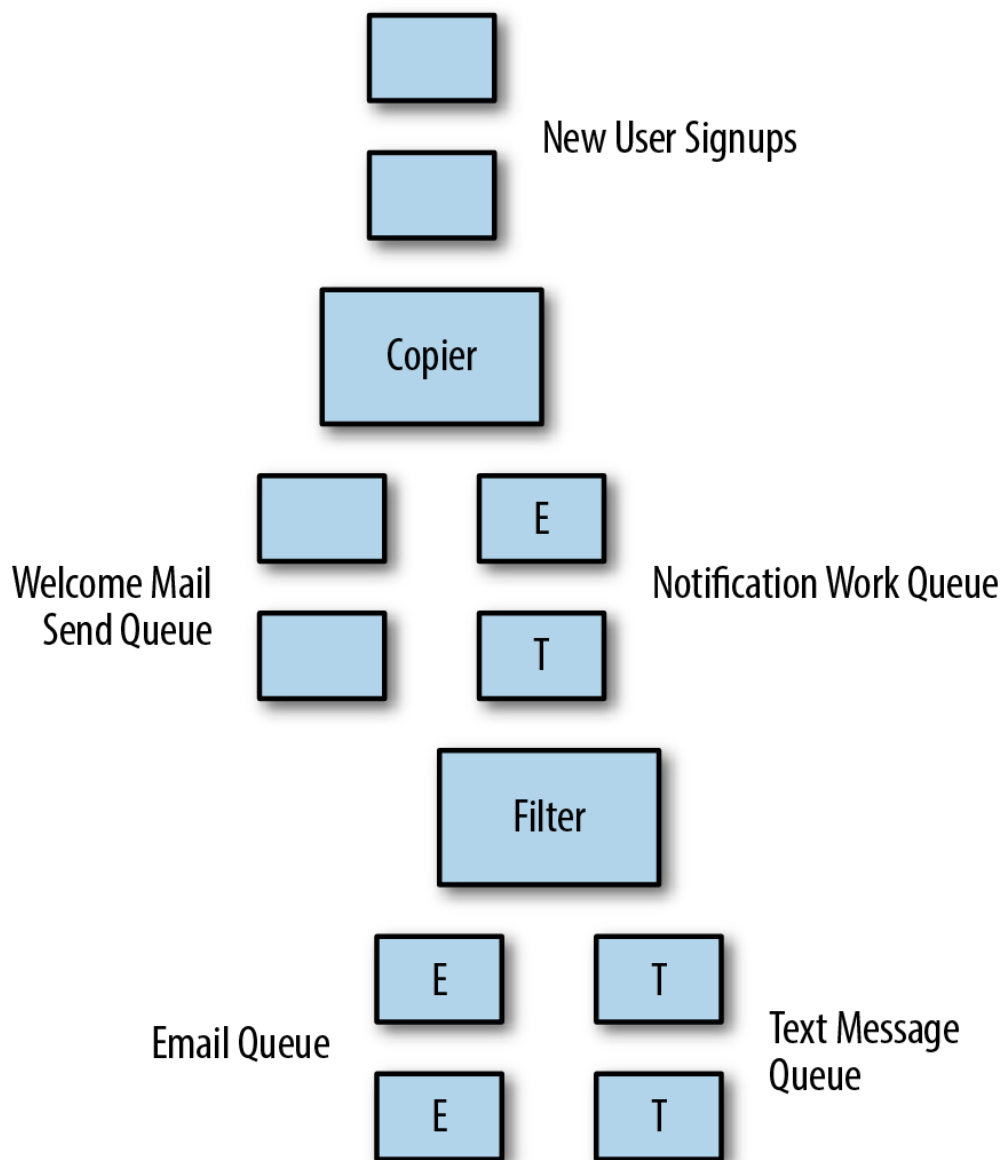


Figure 11-9. The user notification and welcome email work queue

Publisher/Subscriber Infrastructure

We have seen a variety of abstract patterns for linking together different event-driven batch processing patterns. But when it comes time to actually build such a system, we need to figure out how to manage the stream of data that passes through the event-driven workflow. The simplest thing to do would be to simply write each element in the work queue to a particular directory on a local filesystem, and then have each stage monitor that directory for input. But of course doing this with a local filesystem limits our workflow to operating on a single node. We can introduce a network filesystem to distribute files to multiple nodes, but this introduces increasing complexity both in our code and in the deployment of the batch workflow.

Instead, a popular approach to building a workflow like this is to use a publisher/subscriber (pub/sub) API or service. A pub/sub API allows a user to define a collection of queues (sometimes called topics). One or more *publishers* publishes messages to these queues. Likewise, one or

more *subscribers* is listening to these queues for new messages. When a message is published, it is reliably stored by the queue and subsequently delivered to subscribers in a reliable manner.

At this point, most public clouds feature a pub/sub API such as Azure's EventGrid or Amazon's Simple Queue Service. Additionally, the open source [Kafka project](#) provides a very popular pub/sub implementation that you can run on your own hardware as well as on cloud virtual machines. For the remainder of this overview of pub/sub APIs we'll use Kafka for our examples, but they are relatively simple to port to alternate pub/sub APIs.

Hands On: Deploying Kafka

There are obviously many ways to deploy Kafka, and one of the easiest ways is to run it as a container using a Kubernetes cluster and the Helm package manager.

Helm is a package manager for Kubernetes that makes it easy to deploy and manage prepackaged, off-the-shelf applications like Kafka. If you don't already have the `helm` command line tool installed, you can install it from <https://helm.sh>.

Once the `helm` tool is on your machine, you need to initialize it. Initializing Helm deploys a cluster-side component named `tiller` to your cluster and installs some templates to your local filesystem:

```
helm init
```

Now that `helm` is initialized, you can install Kafka using this command:

```
helm repo add incubator http://storage.googleapis.com/kubernetes-charts-incu
helm install --name kafka-service incubator/kafka
```

NOTE

Helm templates have different levels of production hardening and support. `stable` templates are the most strictly vetted and supported, whereas `incubator` templates like Kafka are more experimental and have less production mileage. Regardless, incubator templates are useful for quick proof of concepts as well as a place to start from when implementing a production deployment of a Kubernetes-based service.

Once you have Kafka up and running, you can create a topic to publish to. Generally in batch processing, you're going to use a topic to represent the output of one module in your workflow. This output is likely to be the input for another module in the workflow.

For example, if you are using the `sharder` pattern described previously, you would have a topic for each of the output shards. If you called your output `Photos` and you chose to have three shards, then you would have three topics: `Photos-1`, `Photos-2`, and `Photos-3`. Your `sharder` module would output messages to the appropriate topic, after applying the sharding function.

Here's how you create a topic. First, create a container in the cluster so that we can access Kafka:

```
for x in 0 1 2; do
  kubectl run kafka --image=solsson/kafka:0.11.0.0 --rm --attach --command -
    ./bin/kafka-topics.sh --create --zookeeper kafka-service-zookeeper:2181
    --replication-factor 3 --partitions 10 --topic photos-$x
done
```

Note that there are two interesting parameters in addition to the topic name and the zookeeper service. They are `--replication-factor` and `--partitions`. The replication factor is how many different machines messages in the topic will be replicated to. This is the redundancy that is available in case things crash. A value of 3 or 5 is recommended. The second parameter is the number of partitions for the topic. The number of partitions represents the maximum distribution of the topic onto multiple machines for purposes of load balancing. In this case, since there are 10 partitions, there can be at most 10 different replicas of the topic for load balancing.

Now that we have created a topic, we can send messages to that topic:

```
kubectl run kafka-producer --image=solsson/kafka:0.11.0.0 --rm -it --command
  ./bin/kafka-console-producer.sh --broker-list kafka-service-kafka:9092 \
  --topic photos-1
```

Once that command is up and connected, you should see the Kafka prompt and you can then send messages to the topic(s). To receive messages, you can run:

```
kubectl run kafka-consumer --image=solsson/kafka:0.11.0.0 --rm -it --command
  ./bin/kafka-console-consumer.sh --bootstrap-server kafka-service-kafka:9
```

```
--topic photos-1 \  
--from-beginning
```

Of course, running these command lines only gives you a taste of how to communicate via Kafka messages. To build a real-world event-driven batch processing system, you would likely use a proper programming language and Kafka SDK to access the service. But on the other hand, never underestimate the power of a good Bash script!

This section has shown how installing Kafka into your Kubernetes cluster can dramatically simplify the task of building a work queue based system.

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)